Solution: Combinations

Problem Statement

Given two integers *n* and *k*, return all possible combinations of *k* numbers chosen from the range `[1, n]`.

Each combination must be a unique set of numbers, order of which does not matter.

Examples

- **Example 1**:
  - **Input**: `n = 3, k = 2`
  - **Expected Output**: `[[1, 2], [1, 3], [2, 3]]`
  - **Justification**: `[[1, 2], [1, 3], [2, 3]]` are all combinations of size 2, which we can create using `[1, 2, 3]`.
- **Example 2**:
  - **Input**: `n = 4, k = 1`
  - **Expected Output**: `[[1], [2], [3], [4]]`
  - **Justification**: `[[1], [2], [3], [4]]` are all combinations of size 1, which we can create using `[1, 2, 3, 4].
- **Example 3**:
  - **Input**: `n = 5, k = 3`
  - **Expected Output**: `[[1, 2, 3], [1, 2, 4], [1, 2, 5], [1, 3, 4], [1, 3, 5], [1, 4, 5], [2, 3, 4], [2, 3, 5], [2, 4, 5], [3, 4, 5]]`
  - **Justification**: We unique triplets using numbers 1 to 5, yielding ten different combinations.

Solution

To solve this problem, we can employ a backtracking approach. Backtracking is a recursive strategy for solving problems by exploring all possible solutions and backtrack whenever a solution that's currently being explored no longer seems viable. This approach is particularly effective for generating combinations because it allows us to build combinations one element at a time and backtrack as soon as we've either found a valid combination or need to make a different choice to continue exploring possibilities.

Backtracking works here by choosing a start number for a combination, then recursively choosing the next number, ensuring each number is greater than the previous to maintain uniqueness and avoid duplication. This method efficiently explores all potential combinations of *k* numbers within the given range, ensuring no possibilities are missed and no invalid combinations are included.

Step-by-step Algorithm

1. Initialize an empty list (or array) `result` to store all the combinations.
2. Define a helper function `backtrack` that takes the current combination (`tempList`), the starting number (`start`), 3. `n`, and `k` as parameters. Begin the process by calling `backtrack` with an empty `tempList`, `start` as 1, `n`, and `k`.
3. Check if the current combination's size equals `k`. If so, add a copy of `tempList` to `result` and return.
4. Iterate from the current `start` number to `n`:
   - Add the current number `i` to `tempList`.
   - Recursively call `backtrack` with `i + 1` as the new `start`, to ensure each combination is unique and respects the ascending order.
   - Remove the last number added to `tempList` to backtrack, exploring other possible combinations by trying the next number in the sequence.
5. Once all combinations are explored, return the `result`.

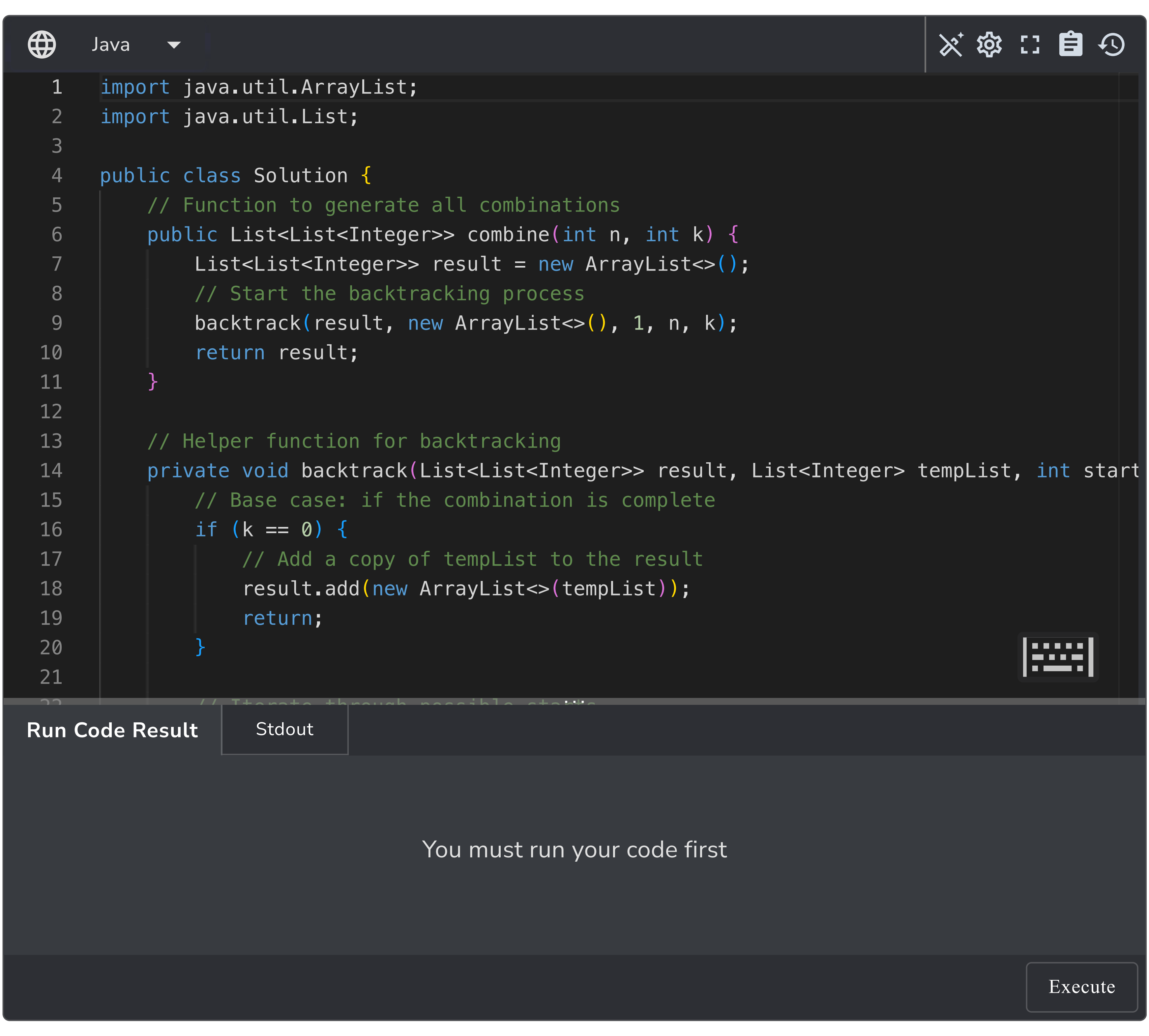
Algorithm Walkthrough

Let's consider the input `n = 5` and `k = 3`:

1. Initialize `result` as an empty list to store combinations.
2. Call `backtrack` with an empty list as `tempList`, `start = 1`, `n = 5`, and `k = 3`.
3. **First Call to Backtrack**:
   - `tempList = []`, `start = 1`
   - Loop from `i = 1` to `5`:
     - Add `1` to `tempList` and call `backtrack` with `start = 2`.
       - **Second Call to Backtrack**:
         - `tempList = [1]`, `start = 2`
         - Loop from `i = 2` to `5`:
           - Add `2` to `tempList` and call `backtrack` with `start = 3`.
             - **Third Call to Backtrack**:
               - `tempList = [1, 2]`, `start = 3`
               - Loop from `i = 3` to `5`:
                 - Add `3` to `tempList` making it `[1, 2, 3]` and since `tempList.size() == k`, add `[1, 2, 3]` to `result`.
                 - Backtrack: Remove `3`, making `tempList = [1, 2]`.
                 - Add `4` to `tempList` making it `[1, 2, 4]` and since `tempList.size() == k`, add `[1, 2, 4]` to `result`.
                 - Backtrack: Remove `4`, making `tempList = [1, 2]`.
                 - Add `5` to `tempList` making it `[1, 2, 5]` and since `tempList.size() == k`, add `[1, 2, 5]` to `result`.
                 - Backtrack: Remove `5`, making `tempList = [1, 2]`.
               - Backtrack to the second call: `tempList = [1]`.
           - Next, `i = 3` in the second call, add `3` to `tempList` making it `[1, 3]` and proceed similarly.
           - Continue this process until all combinations starting with `1` are explored.
     - Backtrack and try next numbers as the starting point, `[2, 3, 4]`, `[2, 3, 5]`, `[2, 4, 5]`, `[3, 4, 5]` will be generated in subsequent calls.
4. After exploring all possible combinations starting from `1` to `5`, the `result` will contain all unique combinations of size `3` from numbers `1` to `5`.
5. Return `result` which now holds all the required combinations: `[[1, 2, 3], [1, 2, 4], [1, 2, 5], [1, 3, 4], [1, 3, 5], [1, 4, 5], [2, 3, 4], [2, 3, 5], [2, 4, 5], [3, 4, 5]]`

   .

Code



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    // Function to generate all combinations
    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> result = new ArrayList<>();
        // Start the backtracking process
        backtrack(result, new ArrayList<>(), 1, n, k);
        return result;
    }

    // Helper function for backtracking
    private void backtrack(List<List<Integer>> result, List<Integer> tempList, int start
        // Base case: if the combination is complete
        if (k == 0) {
            // Add a copy of tempList to the result
            result.add(new ArrayList<>(tempList));
            return;
        }

```

You must run your code first

Complexity Analysis

Time Complexity

- $O(C(n,k)*k)$: The primary factor affecting the time complexity is the number of combinations generated, denoted as `C(n, k)`, where `C(n, k)` is the binomial coefficient representing the number of ways to choose k elements out of n options. For each combination, we perform operations that are proportional to k (to add elements to a temporary list or array). Therefore, the total time complexity is $O(k*C(n,k))$.

Space Complexity

- $O(C(n,k)*k)$: The space required to store all combinations is directly proportional to the number of combinations times the size of each combination, which is k. This accounts for the space needed to store the output.

← Previous
Combinations (Medium)

Next →
Maximum Level Sum Of A Binary Tree (Medium)

☐ Mark as Completed